Close

```
# Preprocess الأنظمة (Baseline legal layer)

> **FIRST STAGE** — Build a structured baseline from 25 Sau

Steps:
1. Extract & unzip law files
2. Parse each law → normalize articles
3. Filter inactive/deleted articles
4. Build unified baseline dataset (CSV + JSONL)
5. Generate baseline statistics & quality report
```

Preprocess الأنظمة (Baseline legal layer)

> **FIRST STAGE** — Build a structured baseline from 25 Saudi commercial laws

Steps:

1. Extract & unzip law files
2. Parse each law → normalize articles
3. Filter inactive/deleted articles
4. Build unified baseline dataset (CSV + JSONL)
5. Generate baseline statistics & quality report

Baseline: Imports & Configuration

```
# BASELINE — IMPORTS & CONFIG
import pandas as pd
import numpy as np
import json
import re
import uuid
import logging
import zipfile
import warnings
from pathlib import Path
from datetime import datetime, timezone

warnings.filterwarnings("ignore")

class BaselineConfig:
    RAW_ZIP_PATH = Path("/content/laws_raw.zip")
    EXTRACT_DIR = Path("/content/laws_extracted")
    OUTPUT_DIR = Path("baseline_output")
    OUTPUT_DIR.mkdir(exist_ok=True, parents=True)

    # Normalization
    REMOVE_DIACRITICS = True
    FILTER_INACTIVE = True  # Remove ملغية/محذوفة articles

# Logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s | %(levelname)s | %(message)s")
bl_logger = logging.getLogger("baseline")

# Arabic patterns
BL_DIACRITICS = re.compile(r'[\u0617-\u061A\u064B-\u0652]')
BL_ALEF = re.compile(r'[إأآا]')
BL_TATWEEL = re.compile(r'ـ+')
BL_NUM_MAP = str.maketrans("٠١٢٣٤٥٦٧٨٩", "0123456789")

def bl_uuid():
    return str(uuid.uuid4())

def bl_normalize_text(text):
    if not text or not isinstance(text, str):
        return ""
    text = BL_TATWEEL.sub("", text)
    if BaselineConfig.REMOVE_DIACRITICS:
        text = BL_DIACRITICS.sub("", text)
    text = BL_ALEF.sub("ا", text)
    text = text.translate(BL_NUM_MAP)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def bl_save_json(data, path):
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def bl_save_jsonl(records, path):
    with open(path, "w", encoding="utf-8") as f:
        for r in records:
            f.write(json.dumps(r, ensure_ascii=False, default=str) + "\n")
```

Baseline Step 1: Extract Law Files

```
# BASELINE STEP 1 — EXTRACT ZIP
def extract_law_files():
    bl_logger.info("Extracting law files from ZIP")
    BaselineConfig.EXTRACT_DIR.mkdir(exist_ok=True, parents=True)

    with zipfile.ZipFile(BaselineConfig.RAW_ZIP_PATH, 'r') as z:
        z.extractall(BaselineConfig.EXTRACT_DIR)

    # Find all JSON files recursively
    json_files = list(BaselineConfig.EXTRACT_DIR.rglob("*.json"))
    bl_logger.info(f"Extracted {len(json_files)} law files")
    for f in json_files:
        bl_logger.info(f"  → {f.name}")
    return json_files

law_files = extract_law_files()
print(f"Total law files: {len(law_files)}")
```

```
Total law files: 25
```

Baseline Step 2: Parse & Normalize Articles

```
# BASELINE STEP 2 — PARSE & NORMALIZE
BASELINE_COLUMNS = [
    "record_id", "law_id", "law_name", "law_info", "law_brief",
    "chapter_name", "article_title", "article_number",
    "article_text_raw", "article_text_clean",
    "status", "is_modified", "modification_details",
    "is_active",
]

def parse_single_law(file_path):
    """Parse one law JSON file into article-level records."""
    with open(file_path, "r", encoding="utf-8") as f:
        data = json.load(f)

    law_id = data.get("lawId", bl_uuid())
    law_name = data.get("lawName", file_path.stem)
    law_info = data.get("info", "")
    law_brief = data.get("brief", "")
    articles = data.get("articles", [])

    records = []
    for art in articles:
        raw_text = art.get("articleText", "") or ""
        # Use newArticleText if available (updated version)
        new_text = art.get("newArticleText", "")
        effective_text = new_text if new_text else raw_text

        status = art.get("status", "سارية")
        is_active = status not in {"ملغية", "محذوفة"}

        records.append({
            "record_id": bl_uuid(),
            "law_id": law_id,
            "law_name": law_name,
            "law_info": law_info,
            "law_brief": law_brief,
            "chapter_name": art.get("chapter", ""),
            "article_title": art.get("articleTitle", ""),
            "article_number": art.get("articleNumber", ""),
            "article_text_raw": effective_text,
            "article_text_clean": bl_normalize_text(effective_text),
            "status": status,
            "is_modified": art.get("isModified", False),
            "modification_details": art.get("modificationDetails", ""),
            "is_active": is_active,
        })
    return records


def build_baseline_dataset(law_files):
    """Parse all law files into one unified dataset."""
    bl_logger.info("Building baseline dataset from all laws")
    all_records = []

    for fp in law_files:
```

```
        try:
            records = parse_single_law(fp)
            bl_logger.info(f"  {fp.name}: {len(records)} articles")
            all_records.extend(records)
        except Exception as e:
            bl_logger.warning(f"  Failed to parse {fp.name}: {e}")

    df = pd.DataFrame(all_records, columns=BASELINE_COLUMNS)
    bl_logger.info(f"Total articles parsed: {len(df)}")
    return df

baseline_raw_df = build_baseline_dataset(law_files)
print(f"Total raw articles: {len(baseline_raw_df)}")
print(f"Laws: {baseline_raw_df['law_name'].nunique()}")
baseline_raw_df.head()
```

```
Total raw articles: 3002
Laws: 25
```

	record_id	law_id	law_name	law_info	law_brief	chapter_name	article_title	article_number	article_tex
0	4e0c3c06-36c1-447a-aacd-66d955985cb2	af2a6b93-51dd-4f16-b781-aafd00d9fbbc	نظام الامتياز التجاري	{'الاسم': 'نظام الامتياز التجاري', 'تاريخ الإص...	نظام الامتياز التجاري 1441 هـ		المادة الأولى	الأولى	عبارات حات الآتية -أينما ب
1	d6bc4edc-121e-427f-9349-fb3d8af77fa1	af2a6b93-51dd-4f16-b781-aafd00d9fbbc	نظام الامتياز التجاري	{'الاسم': 'نظام الامتياز التجاري', 'تاريخ الإص...	نظام الامتياز التجاري 1441 هـ		المادة الثانية	الثانية	ظام إلى تحقيق ما تشجيع أنشطة الا
2	f017c617-e3d2-44f4-8dae-3a6ff53bb1ae	af2a6b93-51dd-4f16-b781-aafd00d9fbbc	نظام الامتياز التجاري	{'الاسم': 'نظام الامتياز التجاري', 'تاريخ الإص...	نظام الامتياز التجاري 1441 هـ		المادة الثالثة	الثالثة	ظام على أي تياز تنفذ داخل
3	ef5da3d5-8302-4d9e-8d8d-ccc86c09d5d7	af2a6b93-51dd-4f16-b781-aafd00d9fbbc	نظام الامتياز التجاري	{'الاسم': 'نظام الامتياز التجاري', 'تاريخ الإص...	نظام الامتياز التجاري 1441 هـ		:المادة الرابعة	الرابعة	تطبيق النظام ، لا باقية امتياز أي

Baseline Step 3: Filter Inactive Articles

```
# BASELINE STEP 3 — FILTER INACTIVE
def filter_baseline(df):
    bl_logger.info("Filtering baseline articles")
    before = len(df)

    # Status distribution before filtering
    status_dist = df["status"].value_counts().to_dict()
    bl_logger.info(f"Status distribution: {status_dist}")

    if BaselineConfig.FILTER_INACTIVE:
        active_df = df[df["is_active"] == True].copy()
    else:
        active_df = df.copy()

    # Remove empty articles
    active_df = active_df[active_df["article_text_clean"].str.len() > 10]

    removed = before - len(active_df)
    bl_logger.info(f"Filtered: {before} → {len(active_df)} (removed {removed})")
    return active_df, status_dist

baseline_df, status_distribution = filter_baseline(baseline_raw_df)
print(f"Active articles: {len(baseline_df)}")
print(f"Status distribution: {status_distribution}")
```

```
Active articles: 2974
Status distribution: {'سارية': 2904, 'معدلة': 63, 'ملغية': 16, 'محذوفة': 12, 'مضافة': 7}
```

Baseline Step 4: Save Baseline Dataset

```
# BASELINE STEP 4 — SAVE OUTPUTS
def save_baseline_outputs(df, status_dist):
    out = BaselineConfig.OUTPUT_DIR

    # 1. Full CSV
    df.to_csv(out / "01_baseline_laws_full.csv", index=False, encoding="utf-8")

    # 2. JSONL (article-level)
    records = df.to_dict(orient="records")
    bl_save_jsonl(records, out / "01_baseline_laws.jsonl")

    # 3. Law-level summary
    law_summary = []
    for law_name, group in df.groupby("law_name"):
        law_summary.append({
            "law_name": law_name,
            "law_id": group["law_id"].iloc[0],
            "total_articles": len(group),
            "chapters": group["chapter_name"].nunique(),
            "modified_articles": int(group["is_modified"].sum()),
        })
    bl_save_json(law_summary, out / "02_law_summary.json")

    # 4. Statistics report
    report = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "total_laws": df["law_name"].nunique(),
        "total_active_articles": len(df),
        "status_distribution": status_dist,
        "laws": {r["law_name"]: r["total_articles"] for r in law_summary},
        "avg_article_length": int(df["article_text_clean"].str.len().mean()),
    }
    bl_save_json(report, out / "03_baseline_report.json")

    # 5. Law names list (for cross-referencing in Phase 6)
    law_names = sorted(df["law_name"].unique().tolist())
    bl_save_json(law_names, out / "04_baseline_law_names.json")

    bl_logger.info(f"Baseline outputs saved to {out}")
    return report

baseline_report = save_baseline_outputs(baseline_df, status_distribution)
print("\nBaseline Report:")
for k, v in baseline_report.items():
    if k != "laws":
        print(f"  {k}: {v}")
print(f"\nLaws ({baseline_report['total_laws']}):")
for name, count in baseline_report["laws"].items():
    print(f"  {name}: {count} articles")
```

```
Baseline Report:
  timestamp: 2026-04-22T20:30:52.691115+00:00
  total_laws: 25
  total_active_articles: 2974
  status_distribution: {'7 :'مضافة' ,12 :'محذوفة' ,16 :'ملغية' ,63 :'معدلة' ,2904 :'سارية'}
  avg_article_length: 317

Laws (25):
  23 : نظام الأسماء التجارية articles
  121 : نظام الأوراق التجارية articles
  129 : نظام الإثبات articles
  222 : نظام الإجراءات الجزائية articles
  231 : نظام الإفلاس articles
  27 : نظام الامتياز التجاري articles
  26 : نظام التجارة الإلكترونية articles
  57 : نظام التحكيم articles
  23 : نظام التكاليف القضائية articles
  97 : نظام التنفيذ articles
  29 : نظام السجل التجاري articles
  281 : نظام الشركات articles
  58 : نظام العلامات التجارية articles
  242 : نظام العمل articles
  85 : نظام القضاء articles
  96 : نظام المحاكم التجارية articles
  55 : نظام المحاماة articles
  225 : نظام المرافعات الشرعية articles
  721 : نظام المعاملات المدنية articles
  81 : نظام المنافسات والمشتريات الحكومية articles
  21 : نظام المنافسة articles
  34 : نظام براءات الاختراع لدول مجلس التعاون لدول الخليج العربية articles
```

```
  40 :نظام مراقبة شركات التمويل articles
  20 :نظام مكافحة التستر articles
  30 :نظام مكافحة الغش التجاري articles
```

Baseline Step 5: Quality Check

```
# BASELINE STEP 5 — QUALITY CHECK
def baseline_quality_check(df):
    bl_logger.info("Running baseline quality check")
    issues = []

    # Empty articles
    empty = df[df["article_text_clean"].str.len() < 20]
    for _, r in empty.iterrows():
        issues.append({
            "issue": "short_article", "law": r["law_name"],
            "article": r["article_number"], "length": len(r["article_text_clean"])
        })

    # Missing article numbers
    missing_num = df[df["article_number"].isna() | (df["article_number"] == "")]
    for _, r in missing_num.iterrows():
        issues.append({
            "issue": "missing_article_number", "law": r["law_name"],
            "article_title": r["article_title"]
        })

    if issues:
        pd.DataFrame(issues).to_csv(
            BaselineConfig.OUTPUT_DIR / "05_baseline_issues.csv",
            index=False, encoding="utf-8"
        )

    bl_logger.info(f"Quality issues found: {len(issues)}")
    return issues

quality_issues = baseline_quality_check(baseline_df)
print(f"Quality issues: {len(quality_issues)}")
print("\n✅ Baseline legal layer complete!")
print(f"   → {baseline_df['law_name'].nunique()} laws")
print(f"   → {len(baseline_df)} active articles")
print(f"   → Saved to: {BaselineConfig.OUTPUT_DIR}")
```

```
Quality issues: 47

✅ Baseline legal layer complete!
   → 25 laws
   → 2974 active articles
   → Saved to: baseline_output
```

Preprocess القضايا (Case layer + chunking + labels)

> **SECOND STAGE — Case Data Processing**

Phases implemented:

1. Data Cleaning & Normalization
2. Text Structuring & Segmentation
3. Legal Entity Recognition & Annotation **(CORRECTED — Hybrid NER)**
4. Tokenization & Embedding Preparation
5. Dataset Balancing & Deduplication
6. Data Validation & Quality Assurance (cross-ref with baseline)

PART 0 — Imports and Configuration

```
# PART 0 — IMPORTS
import pandas as pd
import numpy as np
import json
import re
import uuid
import logging
import random
```

```
import warnings
from pathlib import Path
from collections import defaultdict
from datetime import datetime, timezone

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from bs4 import BeautifulSoup

# NOTE: Transformer NER removed from preprocessing — rules-only is faster
# Transformer NER can be used later during model training (Model A)

warnings.filterwarnings("ignore")
```

```
# CONFIGURATION
class PipelineConfig:
    INPUT_PATH = Path("/content")
    BASELINE_PATH = Path("baseline_output")  # from Stage 1
    OUTPUT_PATH = Path("output")
    OUTPUT_PATH.mkdir(exist_ok=True, parents=True)

    TOKENIZER_NAME = "aubmindlab/bert-base-arabertv02"
    # NER_MODEL_NAME removed — using rules-only in preprocessing
    MAX_SEQUENCE_LENGTH = 512

    REMOVE_DIACRITICS = True
    NORMALIZE_ARABIC_NUMBERS = True
    NEAR_DUPLICATE_THRESHOLD = 0.92

    TRAIN_RATIO = 0.70
    VAL_RATIO = 0.15
    TEST_RATIO = 0.15
    RANDOM_SEED = 42
    # NER_CONFIDENCE_THRESHOLD removed — using rules-only in preprocessing
```

```
# LOGGING & CONSTANTS
LOG_FILE = PipelineConfig.OUTPUT_PATH / "pipeline.log"
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    handlers=[
        logging.FileHandler(LOG_FILE, encoding="utf-8"),
        logging.StreamHandler(),
    ],
)
logger = logging.getLogger(__name__)

UNIFIED_SCHEMA = [
    "case_id", "judgment_number", "court_name", "city", "hijri_date",
    "judgment_text_raw", "appeal_text_raw", "full_case_text_raw",
    "judgment_text_clean", "appeal_text_clean", "full_case_text_clean",
    "source_file", "source_format",
]

SECTION_HEADINGS = [
    "الوقائع", "الأسباب", "الحكم", "الطلبات", "الدفع", "الرد",
    "منطوق الحكم", "نص الحكم", "نص الحكم الابتدائي",
    "أسباب الاستئناف", "الموضوع",
]

ARABIC_DIACRITICS = re.compile(r'[\u0617-\u061A\u064B-\u0652]')
ALEF_VARIANTS = re.compile(r'[إأآا]')
TATWEEL = re.compile(r'_+')
ARABIC_TO_WESTERN = str.maketrans("٠١٢٣٤٥٦٧٨٩", "0123456789")
LEGAL_TERM_STANDARDIZATION = {
    "المدعي": "المدّعي", "المدعى عليه": "المدّعى عليه",
    "المدعيه": "المدّعية", "المدعى عليها": "المدّعى عليها",
}

def generate_uuid():
    return str(uuid.uuid4())
def save_json(data, path):
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)
def save_jsonl(records, path):
    with open(path, "w", encoding="utf-8") as f:
        for r in records:
            f.write(json.dumps(r, ensure_ascii=False, default=str) + "\n")
```

```
# Load baseline law names for Phase 6
def load_baseline_law_names():
    path = PipelineConfig.BASELINE_PATH / "04_baseline_law_names.json"
    if path.exists():
        with open(path, encoding="utf-8") as f:
            return set(json.load(f))
    logger.warning("Baseline law names not found, using defaults")
    return set()

BASELINE_LAW_NAMES = load_baseline_law_names()
logger.info(f"Baseline laws loaded: {len(BASELINE_LAW_NAMES)}")
```

Step 1: Data Cleaning & Normalization

```
# PHASE 1 — DATA CLEANING & NORMALIZATION
def load_raw_cases(input_path):
    records = []
    for file in input_path.glob("*"):
        suffix = file.suffix.lower()
        if suffix not in {".json", ".jsonl", ".csv", ".xlsx", ".xls"}: continue
        logger.info(f"Loading: {file.name}")
        try:
            if suffix == ".json":
                with open(file, encoding="utf-8") as f:
                    data = json.load(f)
                    items = data if isinstance(data, list) else [data]
                    for r in items:
                        r["__source_file"] = file.name
                        r["__source_format"] = "json"
                        records.append(r)
            elif suffix == ".csv":
                df = pd.read_csv(file, encoding="utf-8")
                for _, row in df.iterrows():
                    r = row.to_dict(); r["__source_file"] = file.name; r["__source_format"] = "csv"
                    records.append(r)
            elif suffix in [".xlsx", ".xls"]:
                df = pd.read_excel(file)
                for _, row in df.iterrows():
                    r = row.to_dict(); r["__source_file"] = file.name; r["__source_format"] = "xlsx"
                    records.append(r)
        except Exception as e:
            logger.warning(f"Failed: {file.name}: {e}")
    logger.info(f"Raw records: {len(records)}")
    return records

def normalize_case_schema(record):
    case_id = record.get("id") or generate_uuid()
    jt = record.get("judgmentText") or ""
    at = record.get("appealText") or ""
    return {
        "case_id": case_id, "judgment_number": record.get("judgmentNumber"),
        "court_name": record.get("court"), "city": record.get("city"),
        "hijri_date": record.get("hijriDate"),
        "judgment_text_raw": jt, "appeal_text_raw": at,
        "full_case_text_raw": jt + "\n\n" + at,
        "judgment_text_clean": None, "appeal_text_clean": None,
        "full_case_text_clean": None,
        "source_file": record.get("__source_file"),
        "source_format": record.get("__source_format"),
    }

def remove_html(text):
    return BeautifulSoup(text, "html.parser").get_text()
def normalize_arabic(text):
    text = TATWEEL.sub("", text)
    if PipelineConfig.REMOVE_DIACRITICS: text = ARABIC_DIACRITICS.sub("", text)
    return ALEF_VARIANTS.sub("ا", text)
def clean_single_text(text):
    if not isinstance(text, str) or not text.strip(): return ""
    text = remove_html(text)
    text = re.sub(r"\n?\s*\d+\s*\n", "\n", text)  # page numbers
    text = re.sub(r"[^\w\s\u0600-\u06FF\.\,\:\;\(\)\-\/]", " ", text)  # noise
    text = normalize_arabic(text)
    if PipelineConfig.NORMALIZE_ARABIC_NUMBERS:
        text = text.translate(ARABIC_TO_WESTERN)
    for v, s in LEGAL_TERM_STANDARDIZATION.items():
        text = text.replace(v, s)
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

```
def phase_1_data_cleaning():
    logger.info("=" * 60)
    logger.info("Phase 1 — Data Cleaning & Normalization")
    raw = load_raw_cases(PipelineConfig.INPUT_PATH)
    cleaned = []
    for r in raw:
        c = normalize_case_schema(r)
        c["judgment_text_clean"] = clean_single_text(c["judgment_text_raw"])
        c["appeal_text_clean"] = clean_single_text(c["appeal_text_raw"])
        c["full_case_text_clean"] = (c["judgment_text_clean"] + "\n\n" + c["appeal_text_clean"]).strip()
        cleaned.append(c)
    df = pd.DataFrame(cleaned)
    for col in UNIFIED_SCHEMA:
        if col not in df.columns: df[col] = None
    df = df[UNIFIED_SCHEMA]
    df.to_csv(PipelineConfig.OUTPUT_PATH / "02_cleaned_cases.csv", index=False, encoding="utf-8")
    logger.info(f"Phase 1 done — {len(df)} cases")
    return df
```

```
cleaned_df = phase_1_data_cleaning()
print("Cases:", len(cleaned_df))
cleaned_df.head()
```

```
Cases: 4677
```

	case_id	judgment_number	court_name	city	hijri_date	judgment_text_ra
0	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	4630082310	المحكمة التجارية	الرياض	None	حمد لله والصلاة والسلام على رسول الله :أما ب
1	Gtwi21o7wSn-SSueeVqtaIZlTMnt0N4WymYtp1q4bc1OFp...	4630585068	المحكمة التجارية	الرياض	None	حمد لله والصلاة والسلام على رسول الله، أما ب
2	fkii3nB17yOw8j1iN0rixRxqxtLmi3Yh79c9nV-4KJlxXK...	4630106594	المحكمة التجارية	الرياض	None	حمد لله والصلاة والسلام على رسول الله، أما ب
3	MuO-5FiXHKDWEANdPqiMqtW2BCqmyrh3BeuNFoXZr-FsLv...	4630531487	المحكمة التجارية	الرياض	None	حمدلله والصلاة والسلام على رسول الله أما بعد
4	1FeVZrcOV2LCfbZ5NXMq3rskUxhBBLjpjDXCPGUocFRyKq...	4630454016	المحكمة التجارية	جدة	None	حمد لله والصلاة والسلام على رسول الله أما بع

Step 2: Text Structuring & Segmentation

```
# PHASE 2 — SEGMENTATION (FIXED: Header segment for parties)
def assign_segment_label(s):
    m = {"الوقائع":"Fact","الأسباب":"Reasoning","الحكم":"Verdict","منطوق الحكم":"Verdict",
         "نص الحكم":"Verdict","نص الحكم الابتدائي":"Verdict","الطلبات":"Request",
         "الدفع":"Procedural","الرد":"Procedural","أسباب الاستئناف":"Reasoning","الموضوع":"Fact"}
    return m.get(s, "Other")

def extract_header(text):
    """Extract the header portion of a case (before first section heading).
    This is where plaintiff/defendant names typically appear."""
    esc = [re.escape(h) for h in sorted(SECTION_HEADINGS, key=len, reverse=True)]
    pattern = r'(' + '|'.join(esc) + r')'
    m = re.search(pattern, text)
    if m:
        header = text[:m.start()].strip()
    else:
        header = text[:2000].strip()
    return header

def split_by_section(text):
    esc = [re.escape(h) for h in sorted(SECTION_HEADINGS, key=len, reverse=True)]
    parts = re.split(r"(" + "|".join(esc) + r")\s*[: ]?", text)
    segs, cur = [], None
    for p in parts:
        ps = p.strip()
        if ps in SECTION_HEADINGS: cur = ps; continue
        if ps and len(ps) > 20: segs.append((cur, ps))
    return segs

def phase_2_segmentation(cleaned_df):
    logger.info("=" * 60)
    logger.info("Phase 2 — Segmentation")
    segments = []
    for _, row in cleaned_df.iterrows():
        text = row["full_case_text_clean"]
        if not isinstance(text, str) or len(text) < 50: continue
```

```
        # --- NEW: Extract header as segment 0 (contains parties) ---
        header = extract_header(text)
        if header and len(header) > 10:
            segments.append({
                "segment_id": generate_uuid(),
                "case_id": row["case_id"],
                "segment_order": 0,
                "section_heading": "header",
                "segment_text": header,
                "segment_text_clean": clean_single_text(header),
                "segment_type": "Header",
                "char_count": len(header),
            })

        # --- Regular section splitting ---
        ss = split_by_section(text)
        if not ss: ss = [(None, p.strip()) for p in re.split(r"\n{2,}", text) if len(p.strip()) > 50]
        for i, (sec, st) in enumerate(ss, 1):
            sc = clean_single_text(st)
            if not sc or len(sc) < 10: continue
            segments.append({"segment_id": generate_uuid(), "case_id": row["case_id"],
                "segment_order": i, "section_heading": sec, "segment_text": st,
                "segment_text_clean": sc, "segment_type": assign_segment_label(sec), "char_count": len(sc)})
    seg_df = pd.DataFrame(segments)
    seg_df.to_csv(PipelineConfig.OUTPUT_PATH / "03_segmented.csv", index=False, encoding="utf-8")
    save_jsonl(segments, PipelineConfig.OUTPUT_PATH / "03_segmented.jsonl")
    logger.info(f"Phase 2 done — {len(seg_df)} segments")
    return seg_df
```

```
segmented_df = phase_2_segmentation(cleaned_df)
print("Segments:", len(segmented_df))
segmented_df.head()
```

```
Segments: 114923
```

	segment_id	case_id	segment_order	section_heading	segment_text	segm
0	2b5023c0-3ec4-4de0-a92d-6a6cde1b4cc4	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	0	header	الحمد لله والصلاة والسلام على رسول الله :اما ب...	م على
1	46b111e1-cd6f-473b-9a3a-2009885b758c	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	1	None	الحمد لله والصلاة والسلام على رسول الله :اما ب...	م على
2	0eff93b4-e4a4-4b0d-9315-507474a0870a	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	2	الوقائع	تتلخص وقائع هذه القضية بالقدر اللازم للحكم فيه...	ة

Step 3: Legal Entity Recognition & Annotation

> **CORRECTED** — Hybrid: Transformer NER + Enhanced Rules + Party Extraction

```
# PHASE 3 — ENTITY EXTRACTION (RULES-ONLY, NO TRANSFORMER)
# Transformer NER removed for speed — will be used in Model A training

ENTITY_PATTERNS = {
    "مبلغ_مالي": [r"(?:مبلغ|قدره|بقيمة)\s*(?:وقدره)?\s*[\d٠-٩][,\.\d٠-٩]*\s*ﷺ",
                  r"[\d٠-٩][,\.\d٠-٩]*\s*ﷺ(?:\s*سعودي)?"],
    "مرجع_قانوني": [r"المادة[ه]\s*[(\(]?\s*[\d٠-٩]+\s*[)\)]?(?:\s*من\s+نظام\s+[\u0600-\u06FF\s]+)?",
                    r"الفقرة[ه]\s*[(\(]?\s*[\d٠-٩/]+\s*[)\)]?\s*من\s*المادة[ه]"],
    "اسم_نظام": [r"نظام\s+[\u0600-\u06FF]+(?:\s+[\u0600-\u06FF]+){0,4}",
                 r"اللائحة[ه]\s+للنظام\s+التنفيذي[ه]\s+[\u0600-\u06FF\s]+"],
    "محكمة": [r"المحكمة[ه]\s+(?:العامة[ه]|العليا[ه]|الاستئناف[ه])\s+(?:التجاري)(?:\s+[\u0600-\u06FF]+)?",
              r"الدائرة[ه]\s+(?:الاستئناف[ه]|التجاري[ه])\s+(?:[\u0600-\u06FF\s]+)"],
    "تاريخ": [r"\d{1,2}/\d{1,2}/\d{4}\s*(?:هـ|م|ه)?"],
    "رقم_قضية": [r"(?:الدعوى|القضية)[ه]\s+رقم\s*[\d٠-٩/]+",
```

```
                r"(?:برقم|رقم)\s*[\d٩-٠]+\s*لعام\s*[\d٩-٠]+"],
}

# --- Enhanced party patterns for header extraction ---
PARTY_PATTERNS = [
    # المدعي : اسم  or  المدعي: اسم
    (r"(?:المدّعي|المدّعية)\s*[: /]\s*(.+?)(?:\n|$)", "plaintiff"),
    (r"(?:المدّعى عليه|المدّعى عليها|الموضوع|الوقائع)\s*[: /]\s*(.+?)(?:\n|$)", "defendant"),
    # المدعي\n اسم
    (r"(?:المدّعي|المدّعية)\s*\n\s*(.+?)(?:\n|$)", "plaintiff"),
    (r"(?:المدّعى عليه|المدّعى عليها|الموضوع|الوقائع)\s*\n\s*(.+?)(?:\n|$)", "defendant"),
    # Without diacritics
    (r"المدعي\s+عليه\s*[: ]?\s*\n?\s*(.+?)(?:\n|$)", "defendant"),
    # طرف أول / طرف ثاني
    (r"(?:الطرف الاول|الطرف الأول)\s*[: ]?\s*(.+?)(?:\n|$)", "plaintiff"),
    (r"(?:الطرف الثاني)\s*[: ]?\s*(.+?)(?:\n|$)", "defendant"),
]

PARTY_STOPS = {"بموجب","تقدم","تقدّم","إلى","صحيفة","وحيث","وبعد","تضمنت","تتلخص","تتحصل","الوقائع","الأسباب","حكم"}

def valid_party(n):
    if not n or len(n)<3 or len(n)>200: return False
    if any(n.startswith(s) for s in PARTY_STOPS): return False
    if n in SECTION_HEADINGS: return False
    return True

def clean_party_name(name):
    """Clean extracted party name."""
    name = re.sub(r"\s+", " ", name).strip()
    name = name.rstrip(":  -- .")
    for heading in SECTION_HEADINGS:
        if name.endswith(heading):
            name = name[:name.rfind(heading)].strip()
    return name.strip()

def extract_parties_from_header(full_text, cid):
    """Extract plaintiff & defendant from the FULL case text header.
    Runs on the raw full_case_text, not on segments,
    to catch party names that appear before any section heading."""
    ents = []
    found_roles = set()

    # Search in first 3000 chars (header area)
    search_text = full_text[:3000] if len(full_text) > 3000 else full_text

    for pat, role in PARTY_PATTERNS:
        if role in found_roles: continue
        for m in re.finditer(pat, search_text, re.MULTILINE | re.DOTALL):
            pt = m.group(1).strip().split("\n")[0]
            pt = clean_party_name(pt)
            if valid_party(pt):
                ents.append({
                    "entity_id": generate_uuid(),
                    "case_id": cid,
                    "segment_id": "HEADER",
                    "entity_text": pt,
                    "entity_label": "مدعي" if role == "plaintiff" else "مدعى_عليه",
                    "start_char": m.start(1),
                    "end_char": m.start(1) + len(pt),
                    "extraction_method": "header_rule",
                    "confidence": 0.90,
                })
                found_roles.add(role)
                break
    return ents

def extract_rules(text, cid, sid):
    ents = []
    for label, pats in ENTITY_PATTERNS.items():
        for p in pats:
            for m in re.finditer(p, text):
                et = m.group().strip()
                if len(et)<2: continue
                ents.append({"entity_id":generate_uuid(),"case_id":cid,"segment_id":sid,
                    "entity_text":et,"entity_label":label,"start_char":m.start(),"end_char":m.end(),
                    "extraction_method":"rule_enhanced","confidence":0.80})
    return ents

def dedup_ents(ents):
    seen, out = set(), []
    for e in ents:
        k = (e["case_id"], e["entity_text"], e["entity_label"])
```

```
        if k not in seen: seen.add(k); out.append(e)
    return out

def phase_3_entity_extraction(segmented_df, cleaned_df):
    logger.info("=" * 60)
    logger.info("Phase 3 — Entity Extraction (Rules-Only)")
    all_ents, annots = [], []

    # --- STEP A: Extract parties from FULL case text (header) ---
    logger.info("  Step A: Extracting parties from case headers...")
    party_count = 0
    for _, row in cleaned_df.iterrows():
        full_text = row.get("full_case_text_raw", "") or row.get("full_case_text_clean", "")
        if not isinstance(full_text, str) or len(full_text) < 50: continue
        parties = extract_parties_from_header(full_text, row["case_id"])
        all_ents.extend(parties)
        party_count += len(parties)
    logger.info(f"  Parties extracted from headers: {party_count}")

    # --- STEP B: Extract other entities from segments ---
    logger.info("  Step B: Extracting entities from segments...")
    total = len(segmented_df)
    for idx, (_, row) in enumerate(segmented_df.iterrows()):
        if idx % 500 == 0: logger.info(f"  {idx}/{total}")
        cid, sid, text = row["case_id"], row["segment_id"], row["segment_text_clean"]
        if not isinstance(text, str) or len(text)<10: continue
        se = extract_rules(text, cid, sid)
        all_ents.extend(se)
        annots.append({"case_id":cid, "segment_id":sid, "text":text,
            "entities":[{"start":e["start_char"],"end":e["end_char"],"label":e["entity_label"],"text":e["entity_

    # Deduplicate
    all_ents = dedup_ents(all_ents)

    edf = pd.DataFrame(all_ents)
    edf.to_csv(PipelineConfig.OUTPUT_PATH / "04_entities.csv", index=False, encoding="utf-8")
    save_jsonl(annots, PipelineConfig.OUTPUT_PATH / "14_annotation_ready.jsonl")
    if len(edf)>0:
        save_json({"total":len(edf),"by_label":edf["entity_label"].value_counts().to_dict(),
            "by_method":edf["extraction_method"].value_counts().to_dict()}, PipelineConfig.OUTPUT_PATH / "04_ent

    # --- Log party extraction results ---
    if len(edf) > 0:
        party_df = edf[edf["entity_label"].isin(["مدعي", "مدعى_عليه"])]
        plaintiff_count = len(party_df[party_df["entity_label"] == "مدعي"])
        defendant_count = len(party_df[party_df["entity_label"] == "مدعى_عليه"])
        logger.info(f"  Total parties found: {len(party_df)}")
        logger.info(f"    مدعي: {plaintiff_count}")
        logger.info(f"    مدعى_عليه: {defendant_count}")

    logger.info(f"Phase 3 done — {len(edf)} entities")
    return edf
```

```
entities_df = phase_3_entity_extraction(segmented_df, cleaned_df)
print("Entities:", len(entities_df))
entities_df.head(10)
```

```
Entities: 211032
```

	entity_id	case_id	segment_id	entity_text	entity_label	start_char
0	5bfa477e-20ec-41f4-b6a4-906d236e9cb8	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	HEADER	شركة (...) للمقاولات	مدعي	133
1	c36c0ef8-f0a1-4d6d-bfd7-0f55df57c99f	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	HEADER	شركة (...) للمقاولات (شركة شخص واحد)	مدعى_عليه	168
2	f71e45e5-047f-4b61-9ee5-9d6d127a6d7b	Gtwi21o7wSn-SSueeVqtaIZITMnt0N4WymYtp1q4bc1OFp...	HEADER	(...)	مدعي	137
3	954fd617-f6bb-4078-a2b9-f8f62a15f368	Gtwi21o7wSn-SSueeVqtaIZITMnt0N4WymYtp1q4bc1OFp...	HEADER	(...)	مدعى_عليه	158

```
# Check parties
party_df = entities_df[entities_df["entity_label"].isin(["مدعي","مدعى_عليه"])]
print(f"Parties: {len(party_df)}")
party_df.head(20)
```

```
Parties: 7585
```

	entity_id	case_id	segment_id	entity_text	entity_label	start_char
0	5bfa477e-20ec-41f4-b6a4-906d236e9cb8	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	HEADER	شركة (...) للمقاولات	مدعي	133
1	c36c0ef8-f0a1-4d6d-bfd7-0f55df57c99f	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	HEADER	شركة (...) للمقاولات (شركة شخص واحد)	مدعى_عليه	168
2	f71e45e5-047f-4b61-9ee5-9d6d127a6d7b	Gtwi21o7wSn-SSueeVqtaIZlTMnt0N4WymYtp1q4bc1OFp...	HEADER	(...)	مدعي	137
3	954fd617-f6bb-4078-a2b9-f8f62a15f368	Gtwi21o7wSn-SSueeVqtaIZlTMnt0N4WymYtp1q4bc1OFp...	HEADER	(...)	مدعى_عليه	158
4	3a6e2cc3-0d59-4db3-b7a9-b4e626d1017a	fkii3nB17yOw8j1iN0rixRxqxtLmi3Yh79c9nV-4KJlxXK...	HEADER	(...)	مدعي	131
5	58fbe494-8bb7-4ba3-adfd-a2006f351806	fkii3nB17yOw8j1iN0rixRxqxtLmi3Yh79c9nV-4KJlxXK...	HEADER	(...)	مدعى_عليه	152
6	e00deef7-312f-445c-a6eb-b5be93a432e5	MuO-5FiXHKDWEANdPqiMqtW2BCqmyrh3BeuNFoXZr-FsLv...	HEADER	(...) شركة	مدعي	131
7	76a244b4-5a01-471b-9947-	MuO-5FiXHKDWEANdPqiMqtW2BCqmyrh3BeuNFoXZr-FsLv...	HEADER	(...) شركة	مدعى_عليه	155

Step 4: Tokenization & Embedding Preparation

```
# PHASE 4 — TOKENIZATION
from transformers import AutoTokenizer
import torch

def phase_4_tokenization(segmented_df):
    logger.info("=" * 60)
    logger.info("Phase 4 — Tokenization")
    tokenizer = AutoTokenizer.from_pretrained(PipelineConfig.TOKENIZER_NAME)
    recs = []
    for idx, (_, row) in enumerate(segmented_df.iterrows()):
        if idx % 1000 == 0: logger.info(f"  {idx}/{len(segmented_df)}")
        text = row["segment_text_clean"]
        if not isinstance(text, str) or not text.strip(): continue
        tok = tokenizer(text, padding="max_length", truncation=True,
                        max_length=PipelineConfig.MAX_SEQUENCE_LENGTH, return_tensors="pt")
        actual = int(tok["attention_mask"].sum())
        recs.append({"case_id":row["case_id"],"segment_id":row["segment_id"],
            "segment_type":row.get("segment_type","Other"),
            "input_ids":tok["input_ids"].squeeze().tolist(),
            "attention_mask":tok["attention_mask"].squeeze().tolist(),
            "actual_tokens":actual,"truncated":actual>=PipelineConfig.MAX_SEQUENCE_LENGTH})
    tdf = pd.DataFrame(recs)
    tdf[["case_id","segment_id","segment_type","actual_tokens","truncated"]].to_csv(
        PipelineConfig.OUTPUT_PATH / "05_token_meta.csv", index=False, encoding="utf-8")
    torch.save({"input_ids":torch.tensor([r["input_ids"] for r in recs]),
        "attention_mask":torch.tensor([r["attention_mask"] for r in recs])},
        PipelineConfig.OUTPUT_PATH / "05_tensors.pt")
    logger.info(f"Phase 4 done — {len(tdf)} tokenized, {tdf['truncated'].sum()} truncated")
    return tdf
```

```
token_df = phase_4_tokenization(segmented_df)
print("Tokenized:", len(token_df))
token_df[["case_id","segment_id","actual_tokens","truncated"]].head()
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limi
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
config.json: 100%  384/384 [00:00<00:00, 42.3kB/s]
tokenizer_config.json: 100%  381/381 [00:00<00:00, 46.8kB/s]
vocab.txt:  825k/? [00:00<00:00, 26.8MB/s]
tokenizer.json:  2.64M/? [00:00<00:00, 89.5MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 12.0kB/s]
Tokenized: 114923
```

	case_id	segment_id	actual_tokens	truncated
0	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	2b5023c0-3ec4-4de0-a92d-6a6cde1b4cc4	54	False
1	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	46b111e1-cd6f-473b-9a3a-2009885b758c	54	False
2	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	0eff93b4-e4a4-4b0d-9315-507474a0870a	51	False
3	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	891f97af-1298-448e-8317-0d7f91624fdb	476	False
4	Oo52663n_4unagKCkqMqMPqxqIrp_TBHNN64f2qt9RnGeG...	7ae5f703-66e3-43ed-af85-920e3b8e932a	83	False

Step 5: Dataset Balancing & Deduplication

```
# PHASE 5 — DEDUP & SPLIT
def phase_5_preparation(cleaned_df, segmented_df):
    logger.info("=" * 60)
    logger.info("Phase 5 — Dedup & Split")
    # Exact dedup
    before = len(cleaned_df)
    dedup = cleaned_df.drop_duplicates(subset=["judgment_number"])
    logger.info(f"Exact dups removed: {before - len(dedup)}")
    # Near dedup
    texts = dedup["full_case_text_clean"].fillna("").tolist()
    if len(texts) > 1:
```

```
        try:
            vec = TfidfVectorizer(max_features=5000, analyzer="char_wb", ngram_range=(3,5))
            sim = cosine_similarity(vec.fit_transform(texts))
            rm = set()
            for i in range(len(sim)):
                if i in rm: continue
                for j in range(i+1,len(sim)):
                    if j not in rm and sim[i][j] >= PipelineConfig.NEAR_DUPLICATE_THRESHOLD:
                        rm.add(j)
            if rm:
                logger.info(f"Near-dups: {len(rm)}")
                dedup = dedup[~dedup.index.isin(dedup.index[list(rm)])].reset_index(drop=True)
        except: pass
    # Split
    valid = set(dedup["case_id"].unique())
    sf = segmented_df[segmented_df["case_id"].isin(valid)]
    cids = sf["case_id"].unique()
    tr_c, tmp = train_test_split(cids, test_size=0.30, random_state=42)
    va_c, te_c = train_test_split(tmp, test_size=0.50, random_state=42)
    tr = sf[sf["case_id"].isin(tr_c)]; va = sf[sf["case_id"].isin(va_c)]; te = sf[sf["case_id"].isin(te_c)]
    for n, d in [("06_train",tr),("07_val",va),("08_test",te)]:
        d.to_csv(PipelineConfig.OUTPUT_PATH / f"{n}.csv", index=False, encoding="utf-8")
    rpt = {"train":len(tr),"val":len(va),"test":len(te),"train_cases":len(tr_c),"val_cases":len(va_c),"test_case
    save_json(rpt, PipelineConfig.OUTPUT_PATH / "12_balance.json")
    logger.info(f"Phase 5 done — {rpt}")
    return tr, va, te, rpt
```

```
train_df, val_df, test_df, balance = phase_5_preparation(cleaned_df, segmented_df)
balance
```

```
{'train': 62091,
 'val': 13364,
 'test': 13364,
 'train_cases': 2641,
 'val_cases': 566,
 'test_cases': 567}
```

Step 6: Data Validation & Quality Assurance

> Cross-references entities against 25 baseline laws

```
# PHASE 6 — VALIDATION & QA
CRITICAL_TERMS = ["المدعي","المدعى عليه","المحكمة","الحكم","الوقائع","الأسباب","المادة","النظام"]

def phase_6_validation(cleaned_df, segmented_df, entities_df):
    logger.info("=" * 60)
    logger.info("Phase 6 — Validation & QA")

    # 6.1 Structural
    issues = []
    empty = segmented_df[segmented_df["segment_text_clean"].isna() | (segmented_df["segment_text_clean"].str.str
    for _,r in empty.iterrows():
        issues.append({"type":"empty","case_id":r["case_id"],"segment_id":r["segment_id"]})

    # 6.2 Linguistic
    warns = []
    eseg = set(zip(entities_df["case_id"],entities_df["segment_id"])) if len(entities_df)>0 else set()
    for _,r in segmented_df.iterrows():
        t = r["segment_text_clean"]
        if not isinstance(t,str): continue
        if any(x in t for x in CRITICAL_TERMS) and (r["case_id"],r["segment_id"]) not in eseg:
            warns.append({"type":"term_no_entity","case_id":r["case_id"],"segment_id":r["segment_id"]})

    # 6.3 Legal (cross-ref baseline)
    citations = []
    cpat = re.compile(r"(?:المادة\s*\(?\d+\)?|نظام\s+[\u0600-\u06FF]+(?:\s+[\u0600-\u06FF]+){0,3})")
    for _,r in segmented_df.iterrows():
        t = r["segment_text_clean"]
        if not isinstance(t,str): continue
        for m in cpat.findall(t):
            in_bl = any(law in m for law in BASELINE_LAW_NAMES)
            citations.append({"case_id":r["case_id"],"citation":m,"in_baseline":in_bl})

    report = {"structural":len(issues),"linguistic":len(warns),"citations":len(citations),
              "in_baseline":sum(1 for c in citations if c["in_baseline"])}
    save_json(report, PipelineConfig.OUTPUT_PATH / "11_validation.json")
    if issues+warns:
        pd.DataFrame(issues+warns).to_csv(PipelineConfig.OUTPUT_PATH / "10_problems.csv", index=False, encoding=
    if citations:
```

```
        pd.DataFrame(citations).to_csv(PipelineConfig.OUTPUT_PATH / "10_citations.csv", index=False, encoding="u
    logger.info(f"Phase 6 done — {report}")
    return report
```

```
validation = phase_6_validation(cleaned_df, segmented_df, entities_df)
validation
```

```
{'structural': 0, 'linguistic': 5159, 'citations': 45857, 'in_baseline': 17902}
```

Final: Processing Log & Model-Ready Manifest

```
# FINAL — LOG & MANIFEST
stats = {"cases":cleaned_df["case_id"].nunique(),"segments":len(segmented_df),
         "entities":len(entities_df),"baseline_laws":len(BASELINE_LAW_NAMES)}
save_json({"timestamp":datetime.now(timezone.utc).isoformat(),"version":"2.0",**stats},
          PipelineConfig.OUTPUT_PATH / "13_log.json")
save_json({"dataset":"TMSJ_Saudi_Commercial","status":"APPROVED_FOR_MODEL_TRAINING",
           "timestamp":datetime.now(timezone.utc).isoformat(),**stats},
          PipelineConfig.OUTPUT_PATH / "15_manifest.json")
print("✅ Pipeline complete!")
print(f"   Cases: {stats['cases']}")
print(f"   Segments: {stats['segments']}")
print(f"   Entities: {stats['entities']}")
print(f"   Baseline Laws: {stats['baseline_laws']}")
```

```
✅ Pipeline complete!
   Cases: 4677
   Segments: 114923
   Entities: 211032
   Baseline Laws: 25
```

تجهيز ملفات النماذج الثلاثة (Model-Ready Data)

> هذا القسم يحوّل البيانات المعالجة إلى الصيغ المطلوبة لكل نموذج:
>
> - **Model A** (AraLegal-BERT): NER fine-tuning بصيغة BIO
> - **Model B** (AraGPT-2 + GPT-4o): Scenario generation prompts
> - **Model C** (LLaMA-3 / Aya): Instruction-tuning للتنبؤ بالأحكام

Model A — Reader/Extractor (AraLegal-BERT NER)

> يحتاج بيانات بصيغة **BIO tagging** لتدريب NER على الكيانات القانونية

المخرجات:

- `model_a_ner_bio_train.jsonl` — بيانات التدريب
- `model_a_ner_bio_val.jsonl` — بيانات التحقق
- `model_a_ner_bio_test.jsonl` — بيانات الاختبار
- `model_a_label_map.json` — خريطة التصنيفات

```
from transformers import AutoTokenizer
import torch

# MODEL A — PREPARE NER BIO DATA FOR ARALEGAL-BERT FINE-TUNING

def text_to_bio_tokens(text, entities, tokenizer):
    """
    Convert text + entity spans to BIO-tagged token sequences.
    Each entity gets B-LABEL for first token, I-LABEL for continuation.
    Non-entity tokens get O.
    """
    # Tokenize with offset mapping
    encoding = tokenizer(text, return_offsets_mapping=True, truncation=True,
                         max_length=512, add_special_tokens=True)
    tokens = tokenizer.convert_ids_to_tokens(encoding["input_ids"])
    offsets = encoding["offset_mapping"]

    # Initialize all as O
    bio_tags = ["O"] * len(tokens)

    # Sort entities by start position
    sorted_ents = sorted(entities, key=lambda e: e.get("start", 0))
```

```
    for ent in sorted_ents:
        ent_start = ent.get("start", 0)
        ent_end = ent.get("end", 0)
        label = ent.get("label", "MISC")

        if ent_start >= ent_end:
            continue

        first_token = True
        for idx, (tok_start, tok_end) in enumerate(offsets):
            if tok_start is None or tok_end is None:
                continue
            if tok_start == 0 and tok_end == 0:  # special tokens
                continue

            # Check overlap
            if tok_start >= ent_start and tok_end <= ent_end:
                if first_token:
                    bio_tags[idx] = f"B-{label}"
                    first_token = False
                else:
                    bio_tags[idx] = f"I-{label}"
            elif tok_start < ent_end and tok_end > ent_start:
                # Partial overlap
                if first_token:
                    bio_tags[idx] = f"B-{label}"
                    first_token = False
                else:
                    bio_tags[idx] = f"I-{label}"

    return {
        "tokens": tokens,
        "bio_tags": bio_tags,
        "input_ids": encoding["input_ids"],
        "attention_mask": encoding["attention_mask"],
    }


def prepare_model_a_data(segmented_df, entities_df, train_df, val_df, test_df):
    """Prepare BIO-tagged NER data for Model A fine-tuning."""
    logger.info("=" * 60)
    logger.info("Preparing Model A — NER BIO Data")

    tokenizer = AutoTokenizer.from_pretrained(PipelineConfig.TOKENIZER_NAME)

    # Load annotations
    annot_path = PipelineConfig.OUTPUT_PATH / "14_annotation_ready.jsonl"
    annotations = {}
    if annot_path.exists():
        with open(annot_path, encoding="utf-8") as f:
            for line in f:
                rec = json.loads(line)
                annotations[rec["segment_id"]] = rec

    # Build label set
    all_labels = set()
    for seg_id, annot in annotations.items():
        for ent in annot.get("entities", []):
            all_labels.add(ent["label"])

    label_list = ["O"] + sorted([f"B-{l}" for l in all_labels] + [f"I-{l}" for l in all_labels])
    label_map = {label: idx for idx, label in enumerate(label_list)}

    save_json({"labels": label_list, "label2id": label_map},
              PipelineConfig.OUTPUT_PATH / "model_a_label_map.json")

    # Split case_ids
    train_cases = set(train_df["case_id"].unique())
    val_cases = set(val_df["case_id"].unique())
    test_cases = set(test_df["case_id"].unique())

    splits = {"train": [], "val": [], "test": []}

    for seg_id, annot in annotations.items():
        case_id = annot["case_id"]
        text = annot["text"]
        entities = annot.get("entities", [])

        if not text or len(text) < 10:
            continue
```

```
            bio_record = text_to_bio_tokens(text, entities, tokenizer)
            bio_record["case_id"] = case_id
            bio_record["segment_id"] = seg_id

            if case_id in train_cases:
                splits["train"].append(bio_record)
            elif case_id in val_cases:
                splits["val"].append(bio_record)
            elif case_id in test_cases:
                splits["test"].append(bio_record)

        # Save
        for split_name, records in splits.items():
            path = PipelineConfig.OUTPUT_PATH / f"model_a_ner_bio_{split_name}.jsonl"
            save_jsonl(records, path)
            logger.info(f"  Model A {split_name}: {len(records)} samples → {path}")

        summary = {
            "total_labels": len(label_list),
            "entity_types": sorted(list(all_labels)),
            "train_samples": len(splits["train"]),
            "val_samples": len(splits["val"]),
            "test_samples": len(splits["test"]),
        }
        save_json(summary, PipelineConfig.OUTPUT_PATH / "model_a_summary.json")
        logger.info(f"  Model A data ready: {summary}")
        return summary
```

```
model_a_summary = prepare_model_a_data(segmented_df, entities_df, train_df, val_df, test_df)
model_a_summary
```

```
{'total_labels': 13,
 'entity_types': ['اسم_نظام',
  'تاريخ',
  'رقم_قضية',
  'مبلغ_مالي',
  'محكمة',
  'مرجع_قانوني'],
 'train_samples': 62091,
 'val_samples': 13364,
 'test_samples': 13364}
```

Model B — Scenario Generator (AraGPT-2 + GPT-4o)

> يحتاج:
>
> 1. few-shot examples كـ **أمثلة قضايا حقيقية**
> 2. context للتوليد كـ **مواد الأنظمة**
> 3. **Prompt templates** لتوليد سيناريوهات جديدة

المخرجات:

- `model_b_few_shot_examples.jsonl` — أمثلة قضايا مهيكلة
- `model_b_law_context.jsonl` — مواد الأنظمة كـ context
- `model_b_generation_prompts.jsonl` — prompts جاهزة للتوليد

```
# MODEL B — PREPARE SCENARIO GENERATION DATA

def extract_case_structure(case_row, segmented_df, entities_df):
    """Extract structured representation of a case for few-shot examples."""
    case_id = case_row["case_id"]

    # Get segments for this case
    case_segs = segmented_df[segmented_df["case_id"] == case_id].sort_values("segment_order")

    # Get entities for this case
    case_ents = entities_df[entities_df["case_id"] == case_id] if len(entities_df) > 0 else pd.DataFrame()

    # Extract key parts
    facts = ""
    reasoning = ""
    verdict = ""
    for _, seg in case_segs.iterrows():
        st = seg.get("segment_type", "Other")
        text = seg["segment_text_clean"]
        if st == "Fact":
            facts += text + " "
        elif st == "Reasoning":
```

```
            reasoning += text + " "
        elif st == "Verdict":
            verdict += text + " "

    # Extract key entities
    parties = {"plaintiff": "", "defendant": ""}
    amounts = []
    laws_cited = []

    if len(case_ents) > 0:
        for _, ent in case_ents.iterrows():
            label = ent["entity_label"]
            text = ent["entity_text"]
            if label == "مدعي":
                parties["plaintiff"] = text
            elif label == "مدعى_عليه":
                parties["defendant"] = text
            elif label == "مبلغ_مالي":
                amounts.append(text)
            elif label in ["اسم_نظام", "مرجع_قانوني"]:
                laws_cited.append(text)

    return {
        "case_id": case_id,
        "court": case_row.get("court_name", ""),
        "city": case_row.get("city", ""),
        "plaintiff": parties["plaintiff"],
        "defendant": parties["defendant"],
        "facts_summary": facts[:1000].strip(),
        "reasoning_summary": reasoning[:1000].strip(),
        "verdict_summary": verdict[:500].strip(),
        "amounts": amounts[:5],
        "laws_cited": list(set(laws_cited))[:10],
    }


def prepare_model_b_data(cleaned_df, segmented_df, entities_df, baseline_df=None):
    """Prepare data for AraGPT-2 scenario generation."""
    logger.info("=" * 60)
    logger.info("Preparing Model B — Scenario Generation Data")

    # 1. Few-shot examples (sample diverse cases)
    sample_size = min(200, len(cleaned_df))
    sampled = cleaned_df.sample(n=sample_size, random_state=42)

    examples = []
    for _, row in sampled.iterrows():
        structure = extract_case_structure(row, segmented_df, entities_df)
        if structure["facts_summary"] and structure["verdict_summary"]:
            examples.append(structure)

    save_jsonl(examples, PipelineConfig.OUTPUT_PATH / "model_b_few_shot_examples.jsonl")
    logger.info(f"  Few-shot examples: {len(examples)}")

    # 2. Law context (from baseline)
    law_contexts = []
    bl_path = PipelineConfig.BASELINE_PATH / "01_baseline_laws.jsonl"
    if bl_path.exists():
        with open(bl_path, encoding="utf-8") as f:
            for line in f:
                art = json.loads(line)
                if art.get("is_active", True) and len(art.get("article_text_clean", "")) > 20:
                    law_contexts.append({
                        "law_name": art["law_name"],
                        "chapter": art.get("chapter_name", ""),
                        "article_number": art["article_number"],
                        "article_text": art["article_text_clean"][:500],
                    })
    save_jsonl(law_contexts, PipelineConfig.OUTPUT_PATH / "model_b_law_context.jsonl")
    logger.info(f"  Law context articles: {len(law_contexts)}")

    # 3. Generation prompts
    prompts = []
    case_types = ["إخلال بالعقد", "مطالبة مالية", "نزاع شراكة", "تعويض",
                  "أوراق تجارية", "إفلاس", "علامة تجارية", "فسخ عقد"]

    for ex in examples[:50]:
        for case_type in random.sample(case_types, min(2, len(case_types))):
            prompt = {
                "instruction": f"أنشئ سيناريو قضية تجارية سعودية من نوع: {case_type}",
                "context": {
                    "example_facts": ex["facts_summary"][:300],
```

```
                "laws_available": ex["laws_cited"][:3],
            },
            "expected_output_format": {
                "المدعي": "...",
                "المدعى_عليه": "...",
                "نوع_القضية": case_type,
                "الوقائع": "...",
                "الأسباب_القانونية": "...",
                "المواد_المستند_عليها": ["..."],
                "الحكم_المتوقع": "...",
            },
        }
        prompts.append(prompt)

    save_jsonl(prompts, PipelineConfig.OUTPUT_PATH / "model_b_generation_prompts.jsonl")
    logger.info(f"  Generation prompts: {len(prompts)}")

    summary = {
        "few_shot_examples": len(examples),
        "law_context_articles": len(law_contexts),
        "generation_prompts": len(prompts),
    }
    save_json(summary, PipelineConfig.OUTPUT_PATH / "model_b_summary.json")
    return summary
```

```
import random
model_b_summary = prepare_model_b_data(cleaned_df, segmented_df, entities_df)
model_b_summary
```

```
{'few_shot_examples': 200,
 'law_context_articles': 2973,
 'generation_prompts': 100}
```

Model C — Predictor/Outcome (LLaMA-3 / Aya-Expanse-32B)

> لتدريب نموذج التنبؤ **instruction-tuning** يحتاج بيانات بصيغة:
>
> - **Input**: الوقائع + الأسباب + المواد القانونية المستند عليها
> - **Output**: نص الحكم (الـ verdict)

المخرجات:

- `model_c_instruct_train.jsonl`
- `model_c_instruct_val.jsonl`
- `model_c_instruct_test.jsonl`
- `model_c_label_distribution.json`

```
# MODEL C — PREPARE INSTRUCTION-TUNING DATA FOR OUTCOME PREDICTION
import re

def extract_verdict_from_appeal(appeal_text):
    """Extract the final verdict from appeal text."""
    if not isinstance(appeal_text, str) or len(appeal_text) < 50:
        return None, "unknown"

    last_chunk = appeal_text[-2000:]

    # Try multiple patterns
    patterns = [
        re.compile(r'نص الحكم\s*[: ]?\s*(.{20,600}?)(?:وصلى|وبالله|والله)', re.DOTALL),
        re.compile(r'(?:حكمت|قضت|قررت|المحكمة|الدائرة)\s+(.{10,600}?)(?:وصلى|وبالله|والله)', re.DOTALL),
        re.compile(r'منطوق الحكم\s*[: ]?\s*(.{20,600}?)(?:وصلى|وبالله|والله)', re.DOTALL),
    ]

    verdict_text = None
    for pat in patterns:
        m = pat.search(last_chunk)
        if m:
            verdict_text = m.group(1).strip()
            break

    if not verdict_text:
        # Fallback: last sentence before والله
        m = re.search(r'([^.]{20,300})\s*(?:وبالله|والله)', last_chunk)
        if m:
            verdict_text = m.group(1).strip()

    # Classify
```

```
    if not verdict_text:
        return None, "unknown"

    label = classify_verdict_label(verdict_text)
    return verdict_text, label


def classify_verdict_label(text):
    """Classify verdict text into categories for Model C."""
    if not text:
        return "unknown"

    if 'عدم قبول' in text and ('الاستئناف' in text or 'الاعتراض' in text):
        return "appeal_rejected"
    elif 'تأييد' in text and 'إلزام' in text:
        return "affirmed_obligation"
    elif 'تأييد' in text and 'رفض' in text:
        return "affirmed_rejection"
    elif 'تأييد' in text:
        return "affirmed"
    elif 'إلغاء' in text or 'نقض' in text:
        return "overturned"
    elif 'تعديل' in text:
        return "modified"
    elif 'رفض' in text:
        return "rejected"
    elif 'إلزام' in text:
        return "obligation"
    elif 'انقضاء' in text:
        return "expired"
    else:
        return "other"


def build_instruction_record(case_row, segmented_df, entities_df):
    """Build one instruction-tuning record for Model C."""
    case_id = case_row["case_id"]
    appeal_text = case_row.get("appeal_text_clean", "") or ""
    judgment_text = case_row.get("judgment_text_clean", "") or ""

    # Extract verdict
    verdict_text, verdict_label = extract_verdict_from_appeal(appeal_text)
    if not verdict_text:
        verdict_text, verdict_label = extract_verdict_from_appeal(judgment_text)
    if not verdict_text:
        return None

    # Get structured parts
    case_segs = segmented_df[segmented_df["case_id"] == case_id].sort_values("segment_order")

    facts = ""
    reasoning = ""
    for _, seg in case_segs.iterrows():
        st = seg.get("segment_type", "Other")
        text = seg["segment_text_clean"]
        if st == "Fact":
            facts += text + " "
        elif st == "Reasoning":
            reasoning += text + " "

    if not facts.strip():
        return None

    # Get cited laws
    case_ents = entities_df[entities_df["case_id"] == case_id] if len(entities_df) > 0 else pd.DataFrame()
    laws = []
    if len(case_ents) > 0:
        law_ents = case_ents[case_ents["entity_label"].isin(["اسم_نظام", "مرجع_قانوني"])]
        laws = law_ents["entity_text"].unique().tolist()[:10]

    # Build instruction format
    instruction = "بناءً على وقائع القضية التجارية التالية والأسباب القانونية، تنبأ بالحكم النهائي."

    input_text = f"الوقائع:\n{facts[:1500].strip()}\n\n"
    if reasoning.strip():
        input_text += f"الأسباب:\n{reasoning[:1000].strip()}\n\n"
    if laws:
        input_text += f"المواد المستند عليها:\n{', '.join(laws)}\n"

    return {
        "case_id": case_id,
        "instruction": instruction,
```

```
        "input": input_text.strip(),
        "output": verdict_text[:500].strip(),
        "verdict_label": verdict_label,
        "court": case_row.get("court_name", ""),
        "city": case_row.get("city", ""),
    }


def prepare_model_c_data(cleaned_df, segmented_df, entities_df, train_df, val_df, test_df):
    """Prepare instruction-tuning data for Model C."""
    logger.info("=" * 60)
    logger.info("Preparing Model C — Instruction-Tuning Data")

    train_cases = set(train_df["case_id"].unique())
    val_cases = set(val_df["case_id"].unique())
    test_cases = set(test_df["case_id"].unique())

    splits = {"train": [], "val": [], "test": []}
    label_dist = defaultdict(int)
    skipped = 0

    for _, row in cleaned_df.iterrows():
        record = build_instruction_record(row, segmented_df, entities_df)
        if record is None:
            skipped += 1
            continue

        label_dist[record["verdict_label"]] += 1
        case_id = record["case_id"]

        if case_id in train_cases:
            splits["train"].append(record)
        elif case_id in val_cases:
            splits["val"].append(record)
        elif case_id in test_cases:
            splits["test"].append(record)

    # Save splits
    for split_name, records in splits.items():
        path = PipelineConfig.OUTPUT_PATH / f"model_c_instruct_{split_name}.jsonl"
        save_jsonl(records, path)
        logger.info(f"  Model C {split_name}: {len(records)} samples → {path}")

    summary = {
        "train_samples": len(splits["train"]),
        "val_samples": len(splits["val"]),
        "test_samples": len(splits["test"]),
        "skipped_no_verdict": skipped,
        "label_distribution": dict(label_dist),
    }
    save_json(summary, PipelineConfig.OUTPUT_PATH / "model_c_summary.json")
    logger.info(f"  Model C ready: {summary}")
    return summary
```

```
from collections import defaultdict
model_c_summary = prepare_model_c_data(cleaned_df, segmented_df, entities_df, train_df, val_df, test_df)
print("Model C Summary:")
for k, v in model_c_summary.items():
    print(f"  {k}: {v}")
```

```
Model C Summary:
  train_samples: 2394
  val_samples: 520
  test_samples: 531
  skipped_no_verdict: 374
  label_distribution: {'other': 1832, 'rejected': 1808, 'modified': 98, 'appeal_rejected': 552, 'overturned': 13}
```

ملخص المخرجات النهائية

كل الملفات محفوظة في مجلد `output/`

```
# FINAL — ALL OUTPUTS SUMMARY
print("=" * 70)
print("  TMSJ PREPROCESSING PIPELINE — COMPLETE")
print("=" * 70)
print()

import os
```

```
output_dir = PipelineConfig.OUTPUT_PATH
files = sorted(output_dir.glob("*"))
print(f"📁 Output directory: {output_dir}")
print(f"📄 Total files: {len(files)}")
print()

# Group by model
print("🔷 Baseline (25 Laws):")
for f in sorted(Path("baseline_output").glob("*")):
    size = os.path.getsize(f) / 1024
    print(f"   {f.name:45s} {size:8.1f} KB")

print()
print("🔷 General Pipeline Outputs:")
for f in files:
    if not f.name.startswith("model_"):
        size = os.path.getsize(f) / 1024
        print(f"   {f.name:45s} {size:8.1f} KB")

print()
print("🔷 Model A (AraLegal-BERT NER):")
for f in files:
    if f.name.startswith("model_a"):
        size = os.path.getsize(f) / 1024
        print(f"   {f.name:45s} {size:8.1f} KB")

print()
print("🔷 Model B (AraGPT-2 Scenario Generator):")
for f in files:
    if f.name.startswith("model_b"):
        size = os.path.getsize(f) / 1024
        print(f"   {f.name:45s} {size:8.1f} KB")

print()
print("🔷 Model C (LLaMA-3 / Aya Predictor):")
for f in files:
    if f.name.startswith("model_c"):
        size = os.path.getsize(f) / 1024
        print(f"   {f.name:45s} {size:8.1f} KB")

print()
print("✅ All model-ready files generated successfully!")
```

```
======================================================================
  TMSJ PREPROCESSING PIPELINE — COMPLETE
======================================================================

📁 Output directory: output
📄 Total files: 31

🔷 Baseline (25 Laws):
   01_baseline_laws.jsonl                          7807.7 KB
   01_baseline_laws_full.csv                       7077.1 KB
   02_law_summary.json                                4.7 KB
   03_baseline_report.json                            1.5 KB
   04_baseline_law_names.json                         1.1 KB
   05_baseline_issues.csv                             6.2 KB

🔷 General Pipeline Outputs:
   02_cleaned_cases.csv                          455056.7 KB
   03_segmented.csv                              240934.0 KB
   03_segmented.jsonl                            257717.0 KB
   04_entities.csv                                57164.1 KB
   04_entity_summary.json                             0.3 KB
   05_tensors.pt                                 919385.9 KB
   05_token_meta.csv                              13362.1 KB
   06_train.csv                                  127059.8 KB
   07_val.csv                                     27715.5 KB
   08_test.csv                                    28258.8 KB
   10_citations.csv                                5067.5 KB
   10_problems.csv                                  589.5 KB
   11_validation.json                                 0.1 KB
   12_balance.json                                    0.1 KB
   13_log.json                                        0.2 KB
   14_annotation_ready.jsonl                     169756.3 KB
   15_manifest.json                                   0.2 KB
   pipeline.log                                       0.0 KB

🔷 Model A (AraLegal-BERT NER):
   model_a_label_map.json                             0.7 KB
   model_a_ner_bio_test.jsonl                     42592.6 KB
   model_a_ner_bio_train.jsonl                   194173.7 KB
   model_a_ner_bio_val.jsonl                      42226.3 KB
   model_a_summary.json                               0.3 KB

🔷 Model B (AraGPT-2 Scenario Generator):
```

```
   model_b_few_shot_examples.jsonl                655.6 KB
   model_b_generation_prompts.jsonl               114.3 KB
   model_b_law_context.jsonl                     2028.8 KB
   model_b_summary.json                             0.1 KB

🔷 Model C (LLaMA-3 / Aya Predictor):
   model_c_instruct_test.jsonl                   1815.6 KB
   model_c_instruct_train.jsonl                  8108.9 KB
   model_c_instruct_val.jsonl                    1803.5 KB
   model_c_summary.json                             0.2 KB

✅ All model-ready files generated successfully!
```

حفظ المخرجات على Google Drive

> كل مرة Run عشان ما تحتاجين تعيدين الـ Drive هذا القسم ينسخ **كل** الملفات الناتجة إلى مجلد على

طريقة الاستخدام:

1. شغّلي النوتبوك كامل **مرة وحدة**
2. الملفات تنحفظ تلقائياً على Drive
3. مباشرة Drive تقرأين من Model A/B/C في نوتبوك

```
# MOUNT GOOGLE DRIVE
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# SAVE ALL OUTPUTS TO GOOGLE DRIVE
import shutil

# ======= غيّري المسار هنا حسب مجلدك =======
DRIVE_BASE = "https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link"
# ============================================

DRIVE_BASELINE = f"{DRIVE_BASE}/baseline"
DRIVE_PIPELINE = f"{DRIVE_BASE}/pipeline_output"
DRIVE_MODEL_A = f"{DRIVE_BASE}/model_a_data"
DRIVE_MODEL_B = f"{DRIVE_BASE}/model_b_data"
DRIVE_MODEL_C = f"{DRIVE_BASE}/model_c_data"

# Create all folders
for folder in [DRIVE_BASELINE, DRIVE_PIPELINE, DRIVE_MODEL_A, DRIVE_MODEL_B, DRIVE_MODEL_C]:
    os.makedirs(folder, exist_ok=True)

print("📁 Saving to Google Drive...")
print(f"   Base path: {DRIVE_BASE}")
print()

# --- 1. Baseline files ---
baseline_dir = Path("baseline_output")
if baseline_dir.exists():
    count = 0
    for f in baseline_dir.glob("*"):
        shutil.copy2(f, DRIVE_BASELINE)
        count += 1
    print(f"✅ Baseline: {count} files → {DRIVE_BASELINE}")

# --- 2. Pipeline output files ---
output_dir = PipelineConfig.OUTPUT_PATH
pipeline_files = []
model_a_files = []
model_b_files = []
model_c_files = []

for f in output_dir.glob("*"):
    name = f.name
    if name.startswith("model_a"):
        model_a_files.append(f)
    elif name.startswith("model_b"):
        model_b_files.append(f)
    elif name.startswith("model_c"):
        model_c_files.append(f)
    else:
        pipeline_files.append(f)

# Copy pipeline files
for f in pipeline_files:
```

```
    shutil.copy2(f, DRIVE_PIPELINE)
print(f"✅ Pipeline: {len(pipeline_files)} files → {DRIVE_PIPELINE}")

# Copy Model A files
for f in model_a_files:
    shutil.copy2(f, DRIVE_MODEL_A)
print(f"✅ Model A: {len(model_a_files)} files → {DRIVE_MODEL_A}")

# Copy Model B files
for f in model_b_files:
    shutil.copy2(f, DRIVE_MODEL_B)
print(f"✅ Model B: {len(model_b_files)} files → {DRIVE_MODEL_B}")

# Copy Model C files
for f in model_c_files:
    shutil.copy2(f, DRIVE_MODEL_C)
print(f"✅ Model C: {len(model_c_files)} files → {DRIVE_MODEL_C}")

print()
print("=" * 60)
print("✅ تم حفظ جميع الملفات على Google Drive!")
print("=" * 60)
print()
print("📌 لاستخدام الملفات في نوتبوك ثاني:")
print()
print("   # في بداية نوتبوك Model A:")
print("   from google.colab import drive")
print("   drive.mount('/content/drive')")
print(f"   MODEL_A_PATH = '{DRIVE_MODEL_A}'")
print()
print("   # في بداية نوتبوك Model B:")
print(f"   MODEL_B_PATH = '{DRIVE_MODEL_B}'")
print()
print("   # في بداية نوتبوك Model C:")
print(f"   MODEL_C_PATH = '{DRIVE_MODEL_C}'")
```

```
📁 Saving to Google Drive...
   Base path: https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link

✅ Baseline: 6 files → https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link/ba
✅ Pipeline: 18 files → https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link/p
✅ Model A: 5 files → https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link/moc
✅ Model B: 4 files → https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link/moc
✅ Model C: 4 files → https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link/moc

============================================================
✅ تم حفظ جميع الملفات على Google Drive!
============================================================

📌 لاستخدام الملفات في نوتبوك ثاني:

   # في بداية نوتبوك Model A:
   from google.colab import drive
   drive.mount('/content/drive')
   MODEL_A_PATH = 'https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link/model_

   # في بداية نوتبوك Model B:
   MODEL_B_PATH = 'https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link/model_

   # في بداية نوتبوك Model C:
   MODEL_C_PATH = 'https://drive.google.com/drive/folders/1iQ2EPjkYPrmqs4vlT4ez_QqirCnnHLwy?usp=drive_link/model_
```

نظام الـ Checkpoints (عشان ما تعيدين الرن)

> لو انقطع الاتصال أو حبيتي تكملين بعدين، يبدأ من آخر نقطة Drive لكل مرحلة على **checkpoint** الكود التالي يحفظ

طريقة الاستخدام:

- أول مرة: شغّلي الخلية التالية — تسوي Run كامل وتحفظ checkpoints
- المرات الجاية: شغّلي نفس الخلية — تقرأ الـ checkpoints وتتخطى المراحل المكتملة

```
# CHECKPOINT-ENABLED PIPELINE — RUN THIS CELL ONLY
import pickle

CHECKPOINT_DIR = f"{DRIVE_BASE}/checkpoints"
os.makedirs(CHECKPOINT_DIR, exist_ok=True)

def save_checkpoint(name, data):
    """Save a checkpoint to Drive."""
    path = f"{CHECKPOINT_DIR}/{name}.pkl"
```

```
    with open(path, "wb") as f:
        pickle.dump(data, f)
    print(f"   💾 Checkpoint saved: {name}")

def load_checkpoint(name):
    """Load a checkpoint from Drive. Returns None if not found."""
    path = f"{CHECKPOINT_DIR}/{name}.pkl"
    if os.path.exists(path):
        with open(path, "rb") as f:
            data = pickle.load(f)
        print(f"   ⚡ Checkpoint loaded: {name} — skipping this phase")
        return data
    return None

def clear_all_checkpoints():
    """Clear all checkpoints to force re-run."""
    import glob
    for f in glob.glob(f"{CHECKPOINT_DIR}/*.pkl"):
        os.remove(f)
    print("🗑️ All checkpoints cleared")


print("=" * 60)
print("  TMSJ Pipeline with Checkpoints")
print("=" * 60)
print()

# --- Phase 1 ---
print("📌 Phase 1: Data Cleaning...")
cleaned_df = load_checkpoint("phase1_cleaned")
if cleaned_df is None:
    cleaned_df = phase_1_data_cleaning()
    save_checkpoint("phase1_cleaned", cleaned_df)
print(f"   Cases: {len(cleaned_df)}")
print()

# --- Phase 2 ---
print("📌 Phase 2: Segmentation...")
segmented_df = load_checkpoint("phase2_segmented")
if segmented_df is None:
    segmented_df = phase_2_segmentation(cleaned_df)
    save_checkpoint("phase2_segmented", segmented_df)
print(f"   Segments: {len(segmented_df)}")
print()

# --- Phase 3 ---
print("📌 Phase 3: Entity Extraction...")
entities_df = load_checkpoint("phase3_entities")
if entities_df is None:
    entities_df = phase_3_entity_extraction(segmented_df, cleaned_df)
    save_checkpoint("phase3_entities", entities_df)
print(f"   Entities: {len(entities_df)}")
print()

# --- Phase 4 ---
print("📌 Phase 4: Tokenization...")
token_df = load_checkpoint("phase4_tokens")
if token_df is None:
    token_df = phase_4_tokenization(segmented_df)
    save_checkpoint("phase4_tokens", token_df)
print(f"   Tokenized: {len(token_df)}")
print()

# --- Phase 5 ---
print("📌 Phase 5: Dedup & Split...")
phase5_result = load_checkpoint("phase5_split")
if phase5_result is None:
    train_df, val_df, test_df, balance = phase_5_preparation(cleaned_df, segmented_df)
    phase5_result = {"train": train_df, "val": val_df, "test": test_df, "balance": balance}
    save_checkpoint("phase5_split", phase5_result)
else:
    train_df = phase5_result["train"]
    val_df = phase5_result["val"]
    test_df = phase5_result["test"]
    balance = phase5_result["balance"]
print(f"   Train: {len(train_df)} | Val: {len(val_df)} | Test: {len(test_df)}")
print()

# --- Phase 6 ---
print("📌 Phase 6: Validation...")
validation = load_checkpoint("phase6_validation")
if validation is None:
```

```
    validation = phase_6_validation(cleaned_df, segmented_df, entities_df)
    save_checkpoint("phase6_validation", validation)
print(f"   Report: {validation}")
print()

# --- Model A Data ---
print("📌 Model A: NER BIO Data...")
model_a = load_checkpoint("model_a_ready")
if model_a is None:
    model_a = prepare_model_a_data(segmented_df, entities_df, train_df, val_df, test_df)
    save_checkpoint("model_a_ready", model_a)
print(f"   {model_a}")
print()

# --- Model B Data ---
print("📌 Model B: Scenario Data...")
model_b = load_checkpoint("model_b_ready")
if model_b is None:
    import random
    model_b = prepare_model_b_data(cleaned_df, segmented_df, entities_df)
    save_checkpoint("model_b_ready", model_b)
print(f"   {model_b}")
print()

# --- Model C Data ---
print("📌 Model C: Instruction Data...")
model_c = load_checkpoint("model_c_ready")
if model_c is None:
    from collections import defaultdict
    model_c = prepare_model_c_data(cleaned_df, segmented_df, entities_df, train_df, val_df, test_df)
    save_checkpoint("model_c_ready", model_c)
print(f"   {model_c}")
print()

print("=" * 60)
print("✅ Pipeline complete! All checkpoints saved on Drive.")
print("   Next time you run this cell, completed phases will be skipped.")
print("=" * 60)
```

```
============================================================
  TMSJ Pipeline with Checkpoints
============================================================

📌 Phase 1: Data Cleaning...
   💾 Checkpoint saved: phase1_cleaned
   Cases: 4677

📌 Phase 2: Segmentation...
   💾 Checkpoint saved: phase2_segmented
   Segments: 114923

📌 Phase 3: Entity Extraction...
   💾 Checkpoint saved: phase3_entities
   Entities: 211032

📌 Phase 4: Tokenization...
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_18092/3113395240.py in <cell line: 0>()
     66 token_df = load_checkpoint("phase4_tokens")
     67 if token_df is None:
---> 68     token_df = phase_4_tokenization(segmented_df)
     69     save_checkpoint("phase4_tokens", token_df)
     70 print(f"   Tokenized: {len(token_df)}")

 ─────────────────────────────────────────── 3 frames ──────────────────────────────────────────
/usr/local/lib/python3.12/dist-packages/transformers/tokenization_utils_tokenizers.py in _convert_encoding(self,
encoding, return_token_type_ids, return_attention_mask, return_overflowing_tokens, return_special_tokens_mask,
return_offsets_mapping, return_length, verbose)
    607             if return_offsets_mapping:
    608                 encoding_dict["offset_mapping"].append(e.offsets)
--> 609             if return_length:
    610                 encoding_dict["length"].append(len(e.ids))
    611

KeyboardInterrupt:
```

```
# إذا حبيتي تعيدين مرحلة معينة من الصفر، شغّلي هذا:
# clear_all_checkpoints()  # يمسح كل الـ checkpoints
# أو امسحي checkpoint واحد:
# os.remove(f"{CHECKPOINT_DIR}/phase3_entities.pkl")  # يعيد Phase 3 فقط
```